## Listing 14.11 The modified service relying on external configuration

```
public class ProductionEmailService : IEmailSender
{
    private readonly ILogger<ProductionEmailService> _logger;
    private readonly IConfiguration _config;                                    ❶

    public ProductionEmailService(ILogger<ProductionEmailService>
    ➥ logger, IConfiguration config)                                           ❶
    {
        _logger = logger;
        _config = config;                                                       ❶
    }
    public async Task SendEmailAsync(string email, string subject, string
    ➥ htmlMessage)
    {
        var message = new MimeMessage();
        message.From.Add(MailboxAddress.Parse("test@test.com"));
        message.To.Add(MailboxAddress.Parse(email));
        message.Subject = subject;

        message.Body = new TextPart("html")
        {
            Text = htmlMessage
        };
        var host = _config["SmtpSettings:Host"];                                ❷
        var port = Convert.ToInt32(_config["SmtpSettings:Port"]);               ❷
        var startTls = Boolean.Parse(_config["SmtpSettings:StartTls"]);         ❷
        var userName = _config["SmtpSettings:Credentials:UserName"];            ❷
        var password = _config["SmtpSettings:Credentials:Password"];            ❷
        using var smtp = new SmtpClient();
        await smtp.ConnectAsync(host, port, startTls);
        await smtp.AuthenticateAsync(userName, password);
        await smtp.SendAsync(message);
        await smtp.DisconnectAsync(true);
        _logger.LogInformation("Production email service used");
    }
}
```

❶ The IConfiguration service is injected and assigned to a field.

❷ The configuration values are retrieved using string-based indexes.

This approach works, but as with all APIs that rely on strings, it is error prone. You are a typing mistake away from a `NullReferenceException` at run time. In the next section, you